LABORATORIO DE SEGURIDAD - S-SDLC

Guía del Estudiante

Práctica de las 4 vulnerabilidades web clásicas
Sistema de Gestión Académica (entorno intencionalmente vulnerable)

Unidad: **Seguridad en el Ciclo de Vida del Desarrollo de Software**
Tecnologías: Python + Flask + SQLite + HTML/CSS

> ⚠ **Advertencia ética y legal.** Esta aplicación contiene fallos de seguridad **a propósito**, con fines exclusivamente educativos. Ejecútala **solo en tu equipo local (`127.0.0.1`) o en una máquina virtual aislada**. Las técnicas aquí mostradas solo deben usarse sobre sistemas propios o con autorización explícita. Usarlas contra sistemas ajenos es un delito.

Antes de empezar: cómo levantar el laboratorio

1. Instala Flask: `pip install flask`
2. Crea la base de datos: `python3 database.py`
3. Inicia la versión vulnerable: `python3 app.py` → abre `http://127.0.0.1:5000`
4. (Opcional) En otra terminal, la versión segura: `python3 app_segura.py` → `http://127.0.0.1:5001`

Usuarios de prueba:

Usuario	Contraseña	Rol
admin	admin123	administrador
profesor	clave456	docente
estudiante	1234	estudiante

Ataque 1 — Inyección SQL (SQLi)

¿Qué es? Cuando una aplicación construye sus consultas a la base de datos pegando (concatenando) directamente lo que escribe el usuario, un atacante puede "inyectar" código SQL y cambiar el significado de la consulta.

1A. Saltarse el login sin contraseña

1. Ve a `http://127.0.0.1:5000/login`.
2. En el campo **Usuario** escribe: `admin' --`
3. En **Contraseña** escribe cualquier cosa.
4. Pulsa **Entrar**.

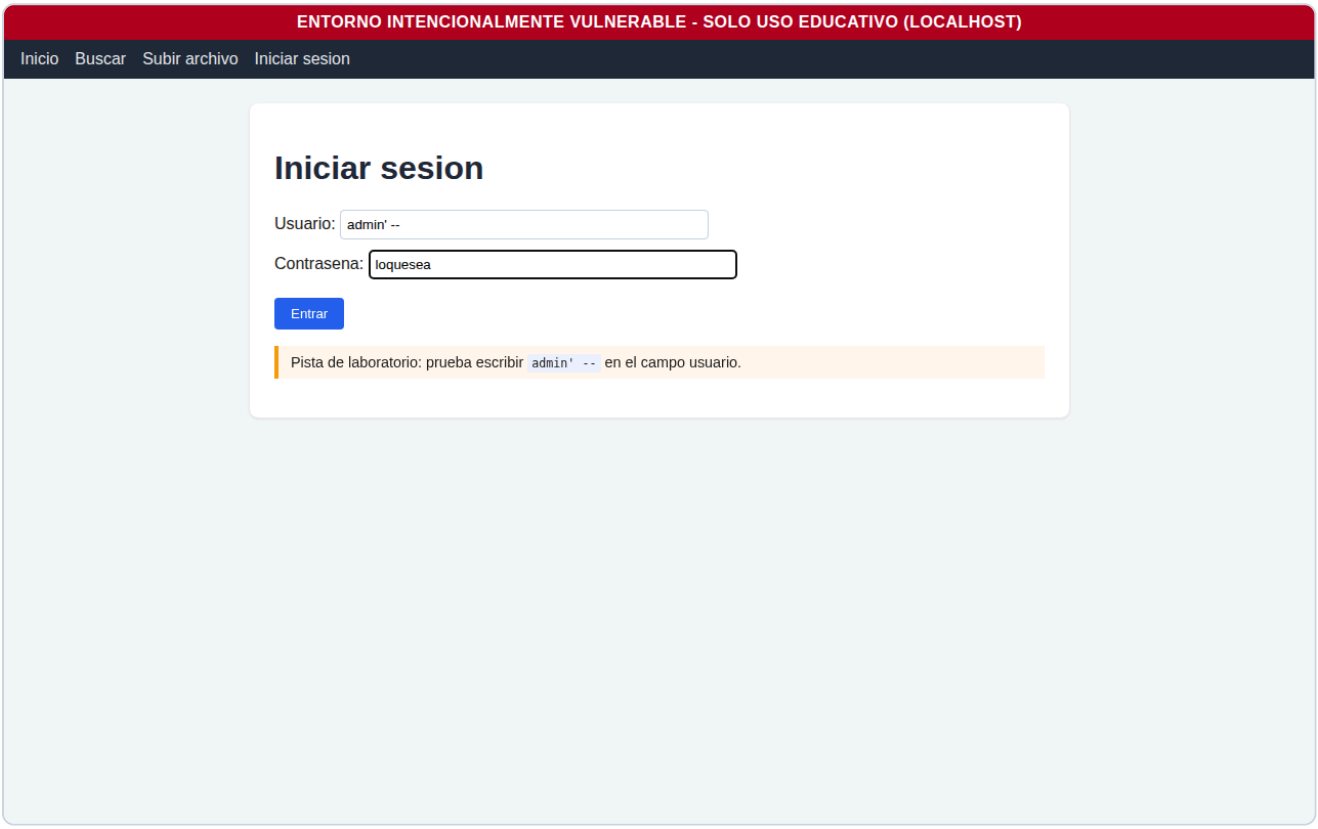


Figura 1.1 — Formulario de login con la inyección `admin' --`.

¿Por qué funciona? La consulta queda así (el `--` comenta el resto, anulando la comprobación de la contraseña):

```
SELECT * FROM usuarios WHERE username = 'admin' --' AND password = '...'
```

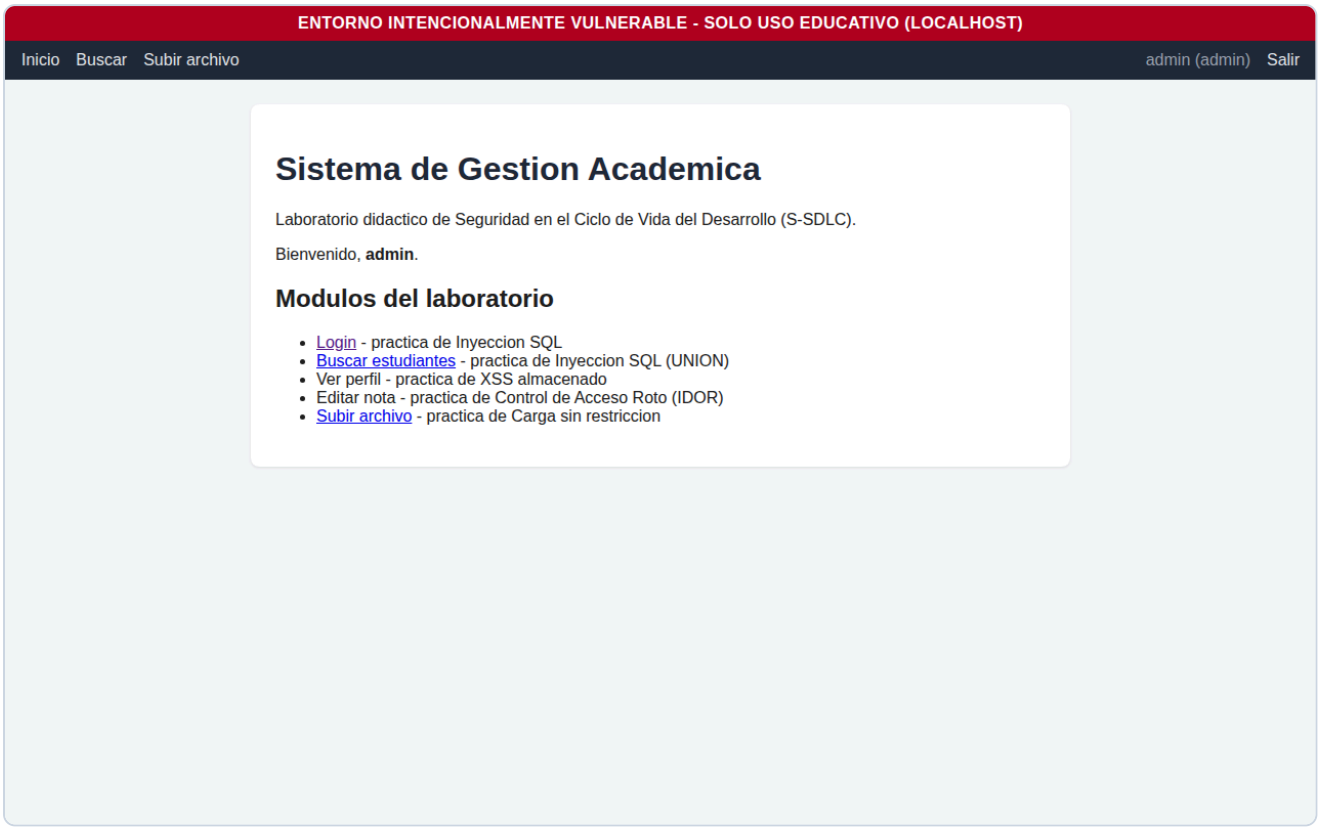


*Figura 1.2 — Sesión iniciada como **admin** sin conocer la contraseña.*

1B. Robar todas las contraseñas con UNION

1. Ve al buscador: `http://127.0.0.1:5000/buscar`.
2. Escribe exactamente: `' UNION SELECT id, username, password, rol FROM usuarios --`
3. Pulsa **Buscar**.

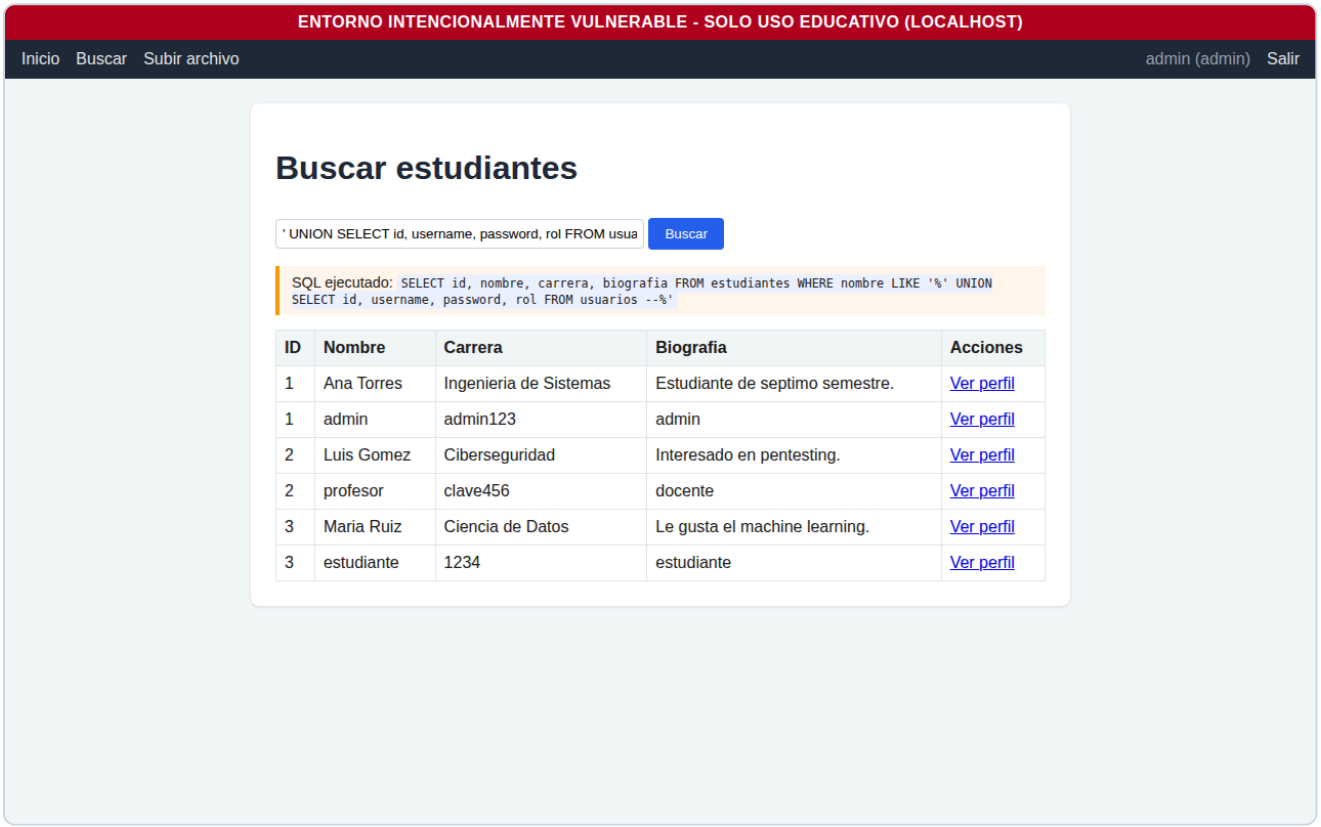


*Figura 1.3 — El ataque UNION mezcla la tabla de usuarios en los resultados: se ven **usuarios y contraseñas en texto plano**.*

Ataque 2 — Cross-Site Scripting (XSS) almacenado

¿Qué es? Si la aplicación guarda texto del usuario y luego lo muestra en la página **sin "escaparlo"**, un atacante puede guardar código HTML/JavaScript que se ejecutará en el navegador de quien vea esa página.

1. Ve al perfil de un estudiante y pulsa **Editar biografia** (o directamente `http://127.0.0.1:5000/perfil/1/editar`).
2. En la biografía pega este código y guarda:

```
<img src=x onerror="this.insertAdjacentHTML('afterend',
'<p style=color:red;font-weight:bold>XSS: JavaScript EJECUTADO</p>')">
```

(También sirve el clásico `<script>alert('XSS')</script>`.)

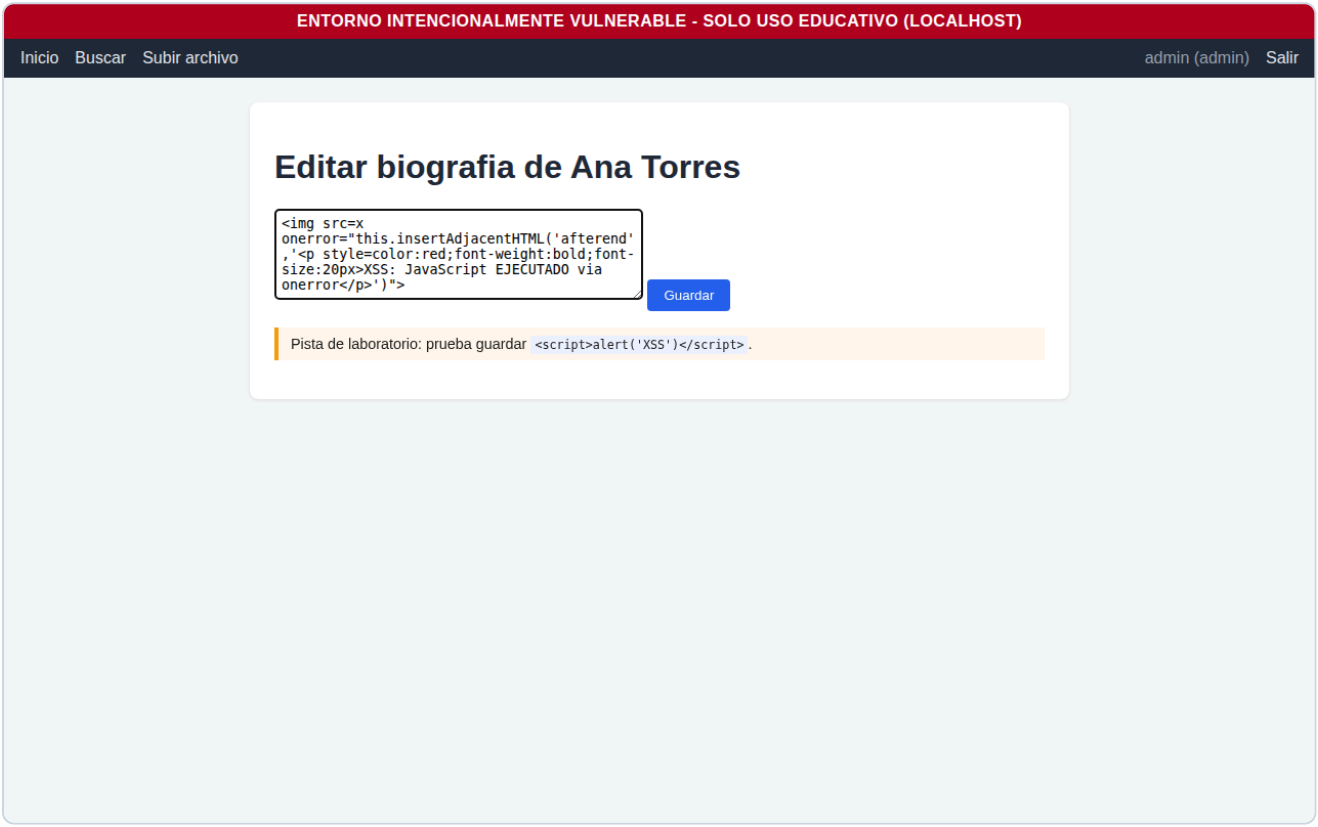


Figura 2.1 — Guardando el código malicioso en el campo biografía.



*Figura 2.2 — Al abrir el perfil, el navegador **ejecuta el código inyectado** (el texto rojo lo generó el JavaScript del atacante).*

Riesgo real: con esto se pueden robar cookies de sesión, suplantar usuarios o redirigir a sitios falsos.

Ataque 3 — Control de acceso roto (IDOR)

¿Qué es? La aplicación identifica un recurso por un número (ID) en la URL, pero **no comprueba si tienes permiso** para modificarlo. Cambiando el número, accedes a datos de otros. Se conoce como *IDOR* (Insecure Direct Object Reference).

1. **Sin iniciar sesión**, ve a `http://127.0.0.1:5000/nota/editar/1`.
2. Verás el formulario para cambiar la nota. Cámbiala y guarda.
3. Prueba con `/nota/editar/2`, `/nota/editar/3` … modificas notas ajenas.

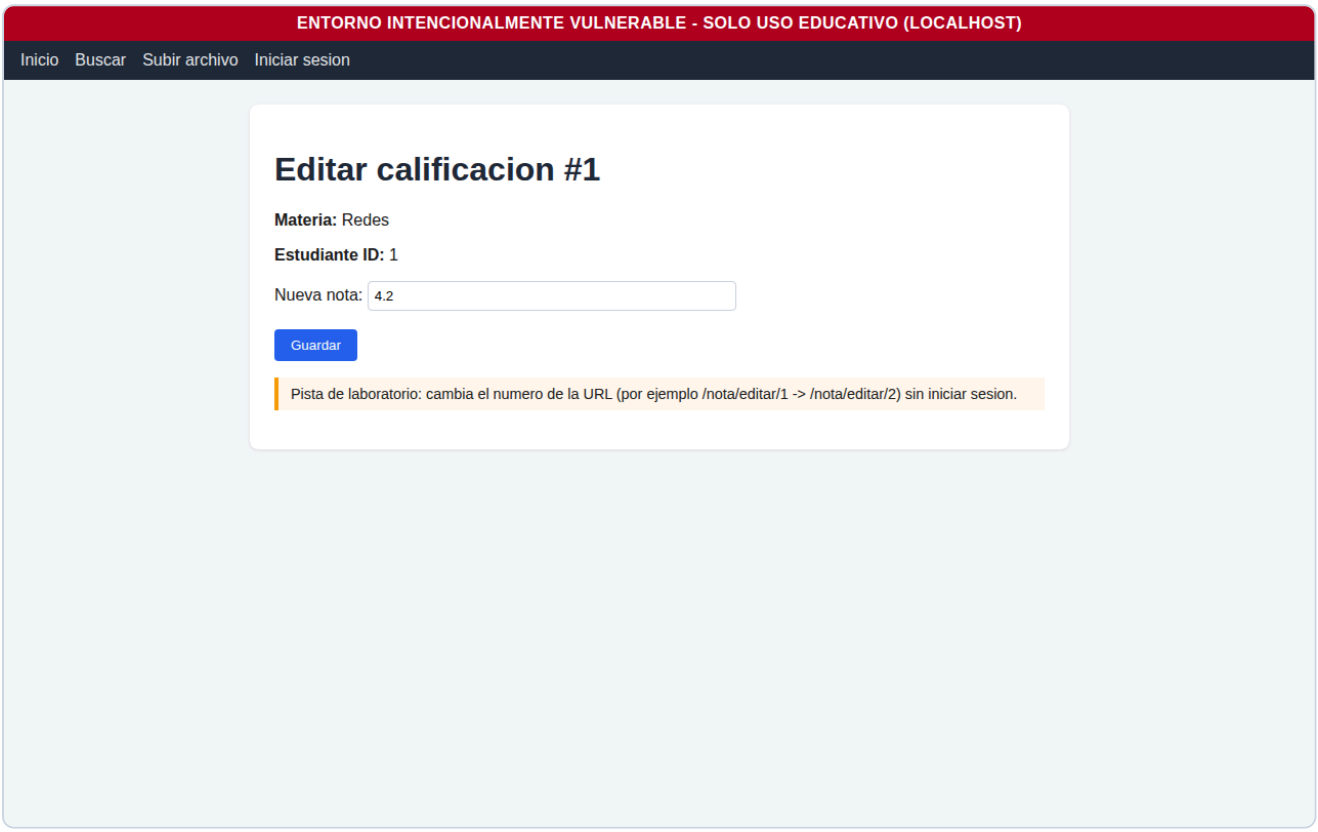


*Figura 3.1 — Cualquiera, **sin haber iniciado sesión**, puede editar una calificación cambiando el número de la URL.*

Ataque 4 — Carga de archivos sin restricción

¿Qué es? La aplicación permite subir **cualquier tipo** de archivo, sin validar la extensión ni el contenido. Un atacante puede subir un script (`.py` , `.php`) y, en un servidor real, llegar a ejecutarlo (RCE).

1. Crea un archivo `malo.py` con cualquier contenido.
2. Ve a `http://127.0.0.1:5000/subir` y súbelo.

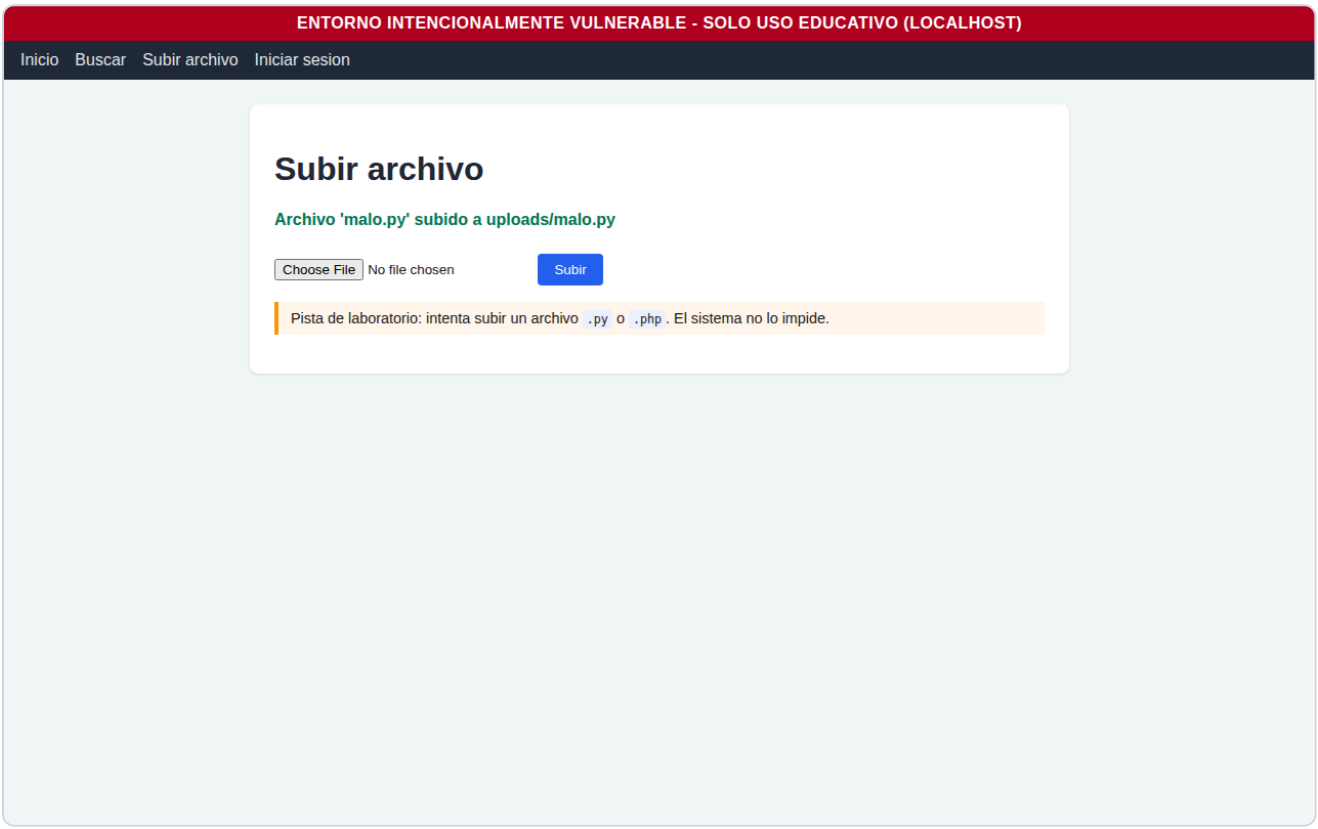


*Figura 4.1 — El sistema **acepta el archivo .py** y confirma que lo guardó en el servidor.*

¿Y la versión segura? (puerto 5001)

Repite los 4 ataques en `http://127.0.0.1:5001`: verás que **todos fallan**. En la siguiente unidad estudiaremos *por qué*. Dos ejemplos:

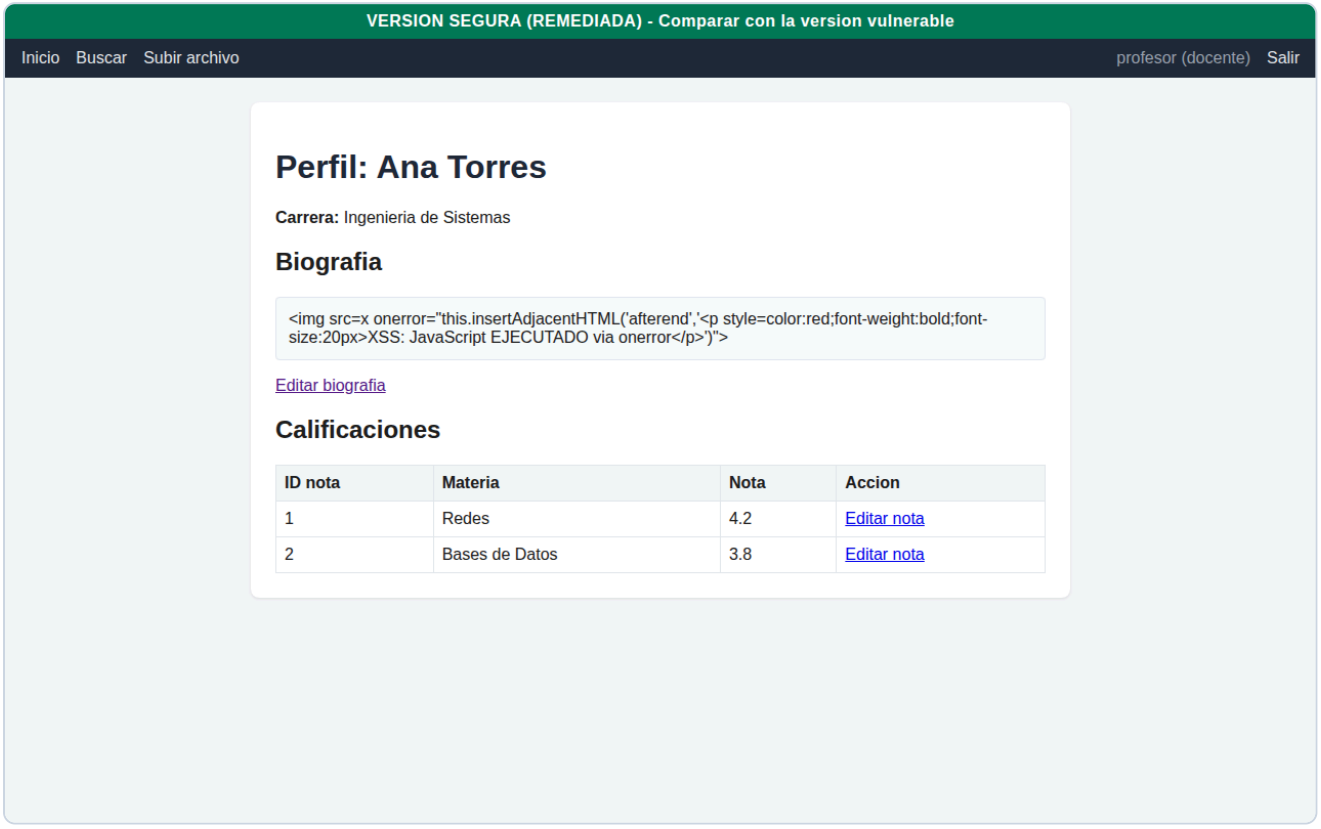


*Figura 5.1 — **XSS bloqueado:** el código se muestra como texto plano, no se ejecuta.*

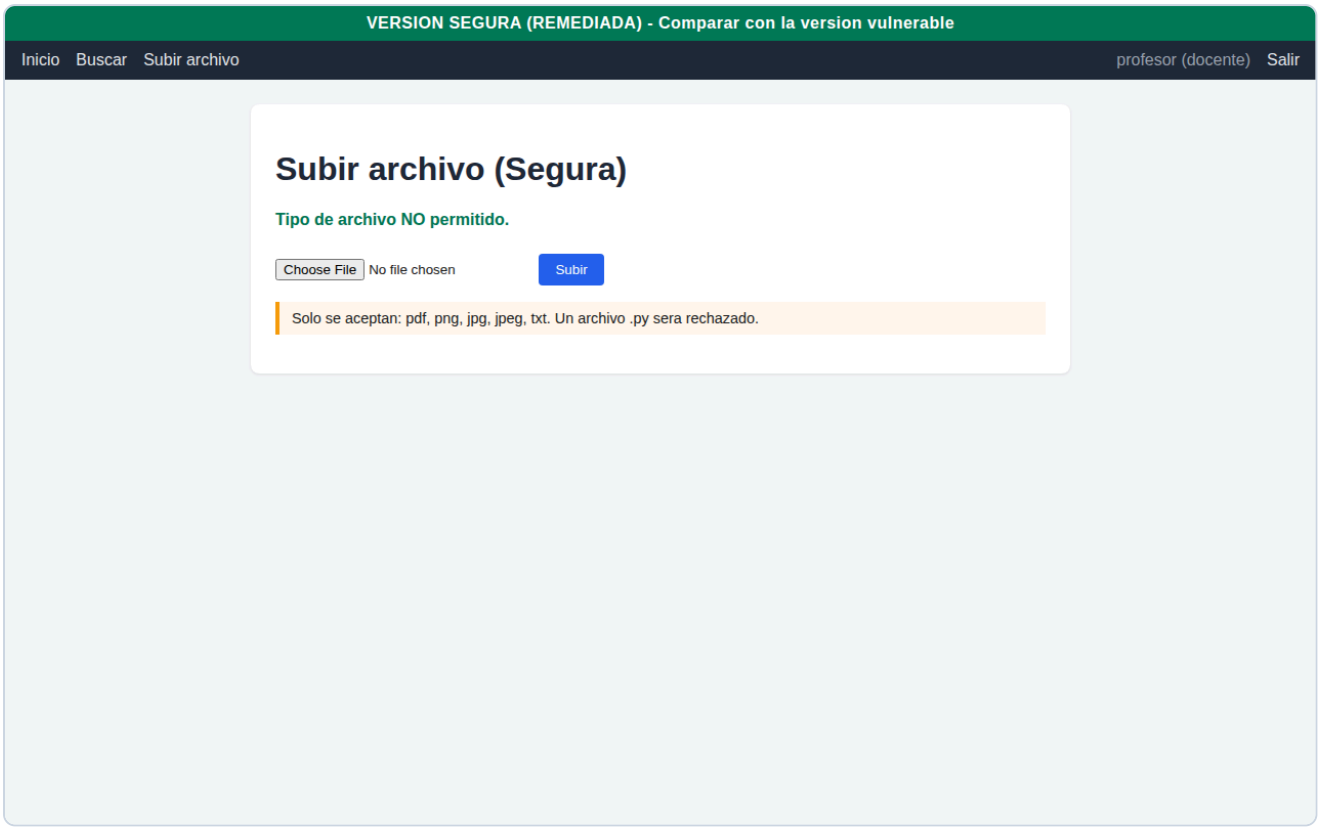


*Figura 5.2 — **Carga bloqueada:** el archivo .py es rechazado.*

Cuestionario de reflexión

1. En el ataque 1A, ¿qué papel cumple el `--` en la inyección?
2. ¿Por qué es tan grave que las contraseñas estén en texto plano (ataque 1B)?
3. En el ataque 2, ¿qué diferencia hay entre XSS "almacenado" y "reflejado"?
4. En el ataque 3, ¿qué comprobación falta en el servidor?
5. En el ataque 4, ¿por qué validar solo la extensión del archivo no siempre es suficiente?